UNIVERSITY OF MALAYA

WIX1002 : FUNDAMENTALS OF PROGRAMMING

VIVA 2: JAVA METHOD

SEM 1 2025 / 2026

LECTURER NAME:

DR. NURUL BINTI JAPAR

GROUP MEMBERS:

1. JASMINE CHIN YING HUI (25005998)
2. NG YUE QHI (25005961)
3. NG SHAO ERN (25006089)
4. NG GEOK LIU (25005946)
5. JASMINE CHIN JIA YEE (25006180)
6. JOSEPHINE DING JIE YU (25005876)

Question 1 : Digital Root Calculator

Section 1 : Problem

The task is to determine the digital root of a positive integer, which is defined as the single-digit value obtained by repeatedly summing its digits. The process involves taking an input and adding its individual digits together to form a new sum. If this new sum still consists of multiple digits, the calculation must be repeated. This iterative cycle continues until a final single-digit result is achieved.

Section 2 : Solution

The program is structured with a specialized method to handle the logic, making the code modular and easy to read. It uses a nested loop approach: the outer loop checks if the number is ten or greater to determine if further reduction is needed, while the inner loop extracts each digit using the modulo operator and accumulates them into a sum. Once all digits are added, the number is updated with the new total. This ensures the program correctly narrows down any large integer to its single-digit digital root before returning the final value to the user.

Section 3 : Sample Input & Output

```
--- exec:3.1.0:exec (default-cli) @ Viva2_Q1 ---
Enter number: 493193
Digital root: 2
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

Test case 1 : Single digit input

```
--- exec:3.1.0:exec (default-cli) @ Viva2_Q1 ---
Enter number: 8
Digital root: 8
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

Test case 2 : Number resulting in 10 during iteration

```
--- exec:3.1.0:exec (default-cli) @ Viva2_Q1 ---
Enter number: 1234
Digital root: 1
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

Section 4 : Source Code

```java
import java.util.Scanner;
public class Viva2_Q1 {
    public static int digitalRoot (int n){
        // Outer loop continues until a single digit remains
        while (n >= 10){
            int sum = 0;
            // Inner loop sums each digit
            while (n > 0){
                sum += n%10;
                n /= 10;
            }
            // Update n with sum of digits
            n = sum;
        }
        return n;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();

        System.out.println("Digital root: "+ digitalRoot(num));
    }
}
```

Question 2: Balanced Parentheses Checker

Section 1 : Problem

The task is to check whether a given string contains **balanced parentheses**.

A string is considered *balanced* if every opening parenthesis `(` has a corresponding closing parenthesis `)` and they appear in the correct order. The string may contain other characters (numbers, letters, operators), but only `(` and `)` are considered for balance checking.

Section 2 : Solution

To solve this problem, I use a **counter-based method:** A counter is created starting at 0. Then, we iterate through the input string character by character. Critical Check: If the counter ever becomes negative during the loop, it means we have encountered a closing parenthesis without a matching open one (e.g., **)(**). In this case, we immediately return `false`. After the loop finishes, we check if the counter is exactly 0. If it is 0, all parentheses were matched correctly. If it is positive, there are unclosed opening parentheses. This logic is implemented inside the `isBalanced` method. The `main` method simply takes user input and calls this method to print the final result.

Section 3 : Sample Input & Output

Set 1 (Balanced Case):

```
Enter expression: (3+5)*(7-(2+1))
Balanced
BUILD SUCCESSFUL (total time: 3 seconds)
```

Set 2 (Not Balanced Case - Missing closing bracket):

```
Enter expression: (5+6*(7-3)
Not balanced
BUILD SUCCESSFUL (total time: 4 seconds)
```

Set 3 (Not Balanced Case - Improper order):

```
Enter expression: )3+5(
Not balanced
BUILD SUCCESSFUL (total time: 2 seconds)
```

Section 4 : Source Code

```java
import java.util.Scanner;
public class Q2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Prompt user for input
        System.out.print("Enter expression: ");
        String input = sc.nextLine();

        // Call the method and print the result
        if (isBalanced(input)) {
            System.out.println("Balanced");
        } else {
            System.out.println("Not balanced");
        }
    }
    // Method to check if parentheses are balanced
    public static boolean isBalanced(String s) {
        int count = 0;
        // Iterate over the string length
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (ch == '(') {
                count++;
            } else if (ch == ')') {
                count--;

                // If count becomes negative, closing comes before opening
                if (count < 0) {
                    return false;
                }
            }
        }

        // Return true only if count returns to exactly zero
        return count == 0;
    }
}
```

Question 3: Lucky Ticket Verifier

Section 1 : Problem

The task is to determine whether a given ticket number is lucky, unlucky or invalid.

The program must validate whether the string contains non-digit characters or if the string length is odd, If either condition is true, the program will print “Invalid ticket number.”

If the ticket is valid, the program will compare the sum of the first half and the second half of the digits. If the sums are equal, “Lucky” will be printed; otherwise, “Unlucky” will be printed.

Section 2 : Solution

First, the program prompts the user to enter a ticket number as a string.

The program checks whether the string contains only digits using the isDigit method. It also checks whether the length of the string is even. If either condition fails, the program prints "Invalid ticket number." and terminates.

If the input is valid, the program determines the midpoint (half) of the string by dividing its length by 2. The first half consists of the digits in positions 0 to half - 1, and the second half consists of the digits in positions half to the end of the string.

Next, the program calculates the sum of digits in the first half and the second half using loops. Each character is converted to its numeric value by subtracting '0'.

The program then compares the two sums. If the sums are equal, the program prints "Lucky". Otherwise, the program prints "Unlucky".

Section 3 : Sample Input & Output

```
Enter ticket number: 124A23
Invalid ticket number.
BUILD SUCCESSFUL (total time: 4 seconds)
```

```
Enter ticket number: 152125
Lucky
BUILD SUCCESSFUL (total time: 3 seconds)
```

```
Enter ticket number: 133152
Unlucky
BUILD SUCCESSFUL (total time: 3 seconds)
```

```
Enter ticket number:
Invalid ticket number.
BUILD SUCCESSFUL (total time: 1 second)
```

Section 4 : Source Code

```java
import java.util.Scanner;

public class Q3 {
    public static Boolean isLuckyTicket(String ticket) {
        int left = 0, right = 0;
        int half = ticket.length() / 2;

        for (int i = 0; i < half; i++) {
            left += ticket.charAt(i) - '0';
        }

        for (int i = half; i < ticket.length(); i++) {
            right += ticket.charAt(i) - '0';
        }

        return left == right;
    }

    public static boolean isDigit(String ticket) {
        for (int i = 0; i < ticket.length(); i++) {
            char ch = ticket.charAt(i);
            if (!Character.isDigit(ch))
                return false;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter ticket number: ");
        String ticketNum = sc.nextLine();

        if (ticketNum.isEmpty() || !isDigit(ticketNum) || ticketNum.length() % 2 != 0)
            System.out.println("Invalid ticket number.");
        else {
            if (isLuckyTicket(ticketNum))
                System.out.println("Lucky");
            else
                System.out.println("Unlucky");
        }
    }
}
```

Question 4: Tic-Tac-Toe Winner Checker

Section 1 : Problem

This question required us to determine the winner in a completed 3x3 Tic-Tac-Toe board. The program reads three input lines, which is 3 rows of the board and each row containing exactly three characters which can only consist of 'X' (Player X), 'O' (Player O) or '.' (empty square).

There are some requirements to be fulfilled to prove that the moves are valid:

- Since player X starts first, the number of X should be equal or one more than number of O
- Only characters 'X'/ 'O'/ '.' are accepted
- 1 row should only consists of 3 characters

If the moves are valid, the winner of the game can be determined if one of the players has the same symbol appears in any complete row, column, or one of the two diagonals. If none of them wins, there will be no winner for the game.

Section 2 : Solution

There will be 3 methods:

- main method to received inputs which fulfill the requirement about the types of character entered and number of characters in each row
- checkWinner method to return the result of the game
- countMoves method to count the number of 'X' and 'O' to make sure the moves are valid

In the main method:

1. Read the three board rows from the user. Loop through every row to make sure there are only 3 characters per row and the character only consists of 'X'/ 'O'/ '.'
2. Construct a 2D array of characters.
3. Call countMoves() for each character 'X'/ 'O' and compare the number of character 'X'/ 'O' to determine whether the moves are valid
4. Call checkWinner() to determine the result.
5. Display the output as:

a. "Winner: X" or "Winner: O" if a player has won,

b. "No winner" if the game has no winning line.

c. Invalid board: number of moves is not valid. If the number of O is greater than number of X

In the checkWinner method with the (char[][] board) as parameter:

1. Returns 'X' if player X has won,'O' if player O has won, or '.' if there is no winner.

In the countMoves with the (char[][] board, char player) as parameters:

1. Returns the total number of moves made by the specified player(‘X’/ ‘O’).

Section 3 : Sample Input & Output

```
Enter row 1
XXX
Enter row 2
0.0
Enter row 3
XOX
Invalid board: number of moves is not valid.
```

```
Enter row 1
XO.
Enter row 2
OX.
Enter row 3
..O
Invalid board: number of moves is not valid.
```

```
Enter row 1
XOXO
You are only allowed to enter 3 character in a row
Enter row 1
XOP
You enter an invalid character. Please enter X/O/.:
Enter row 1
XOO
Enter row 2
.X.
Enter row 3
..X
Winner: X
```

Section 4 : Source Code

```
import java.util.Scanner;
public class viva2q4 {
    public static void main(String args[]) {
        Scanner sc=new Scanner(System.in);
        char[][]board=new char[3][3];
        for(int i=0;i<3;i++){
            String row="";
            boolean validRow=false;
            //loop continuosly until we get a valid row
            while(!validRow){
                System.out.println("Enter row "+(i+1));
                row=sc.nextLine().trim();
                if(row.length()!=3){
                    System.out.println("You are only allowed to enter 3 character in a row");
                    continue;}
                validRow=true;
                for(int j=0;j<3;j++){
                char result=row.charAt(j);
                //loop through the row to check every characters
                while(result!='X'&&result!='O'&&result!='.'){
                    System.out.println("You enter an invalid character. Please enter X/O/.:");
                    validRow=false;
                    //stop checking this string and repeat again
                    break;}}}
            for(int j=0;j<3;j++){
                board[i][j]=row.charAt(j);
                }}
        int countX=countMoves(board,'X');
        int countO=countMoves(board,'O');
        if((countO>countX)||((countX-countO)>1))
            System.out.println("Invalid board: number of moves is not valid.");
        else{
            if(checkWinner(board)=='.')
                System.out.println("No Winner");
            else
                System.out.println("Winner: "+checkWinner(board));}}
```

```
public static char checkWinner(char[][]board){
    char player1='X',player2='O';
    for(int i=0;i<board.length;i++){
        if((board[i][0]=='X'&&board[i][1]=='X'&&board[i][2]=='X')||(board[0][i]=='X'&&board[1][i]=='X'&&board[2][i]=='X'))
            return player1;
        else if((board[i][0]=='O'&&board[i][1]=='O'&&board[i][2]=='O')||(board[0][i]=='O'&&board[1][i]=='O'&&board[2][i]=='O'))
            return player2;
        }
        if ((board[0][0]=='X'&&board[1][1]=='X'&&board[2][2]=='X')||(board[0][2]=='X'&&board[1][1]=='X'&&board[2][0]=='X'))
            return player1;
        else if ((board[0][0]=='O'&&board[1][1]=='O'&&board[2][2]=='O')||(board[0][2]=='O'&&board[1][1]=='O'&&board[2][0]=='O'))
            return player2;

    return '.';
}
public static int countMoves(char[][]board,char player){
    int count=0;
    for (char[] board1 : board) {
        //board.length=num of rows
        for (int j = 0; j < board1.length; j++) {
            if (board1[j] == player) {
                count++;
            }
        }
    }
    return count;
}}
```

Question 5 : Run-Length Encoding (RLE) Compressor / Decompressor

Section 1 : Problem

For the compressor of a String with repeated characters, we shall convert it from a consecutive character into format of <count><char>.We shall also take spaces and other symbols into consideration as long as it is not a digit.

For the decompressor, we receive a String of compressed message from the user.Based on the message, we shall expand it back to the original String. This conversion is limited with certain rules , Each digit must be a positive integer, Each digit shall not be followed by another digit. For successful operation, the results shall be returned and printed. For invalid operation, the results will not be returned and an error message will be printed.

Section 2 : Solution

First of all, we shall create a main class that has 3 methods, A main method, A compressor method, and A decompressor method.

In the main method, we will receive the Mode (C / D) requested by the user and a Text to carry out the selected method. The main method will also be used to print results or error message returned by the other methods.

In the compressor method , the String text will be split into a character array. Then we shall first ensure that the text does not contain any digit. If a digit is detected , the method will print an error message and stop.After prior checking, we shall use a for loop to calculate the count of that certain character until a new character occurs. When a new character occurs , we shall add the count of the previous character and the character into a String result , which will be returned at the end of method.

In the decompressor method, the String text will be split into a character array.Then we shall first ensure the first character is a digit and the last character is not a digit. If the check is not passed, an error message will be returned and the method shall end. If a check is passed, we will loop through characters in the array . If the current character is a digit , we shall make sure that it is not continued by a digit. If this check is not passed , the error message will be returned and the method shall end. If a character is other than a digit , we shall add the character for count based on the digit before it into the result. The String result will be returned to the main method.

Section 3 : Sample Input & Output

```
run:
Mode (C / D) : C
Text : A  BBB
Result :1A2 3B
BUILD SUCCESSFUL (total time: 5 seconds)
```

```
run:
Mode (C / D) : D
Text : 1A2 3B
Result :A  BBB
BUILD SUCCESSFUL (total time: 6 seconds)
```

```
run:
Mode (C / D) : D
Text : 12
Result :Invalid Encoding
BUILD SUCCESSFUL (total time: 3 seconds)
```

Section 4 : Source Code

```java
import java.util.*;
public class Q5 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Mode (C / D) : ");
        char mode = input.next().charAt(0);
        input.nextLine();
        System.out.print("Text : ");
        String text = input.nextLine();

        switch (mode) {
            case 'C' -> System.out.println("Result :" +compress(text));
            case 'D' -> System.out.println("Result :" +decompress(text));
            default -> System.out.println("Invalid Mode");
        }
    }
```

```java
    public static String compress(String s){
        String result= "";

        char[] list = s.toCharArray();

        for(char c : list){
            if (Character.isDigit(c)){
                result = "Invalid Text";
                return result;
            }
        }

        int count = 1;
        char current = list[0];

        for(int i = 1; i < list.length; i++){
            if(list[i] == current){
                count ++;
            }else{
                result += count;
                result += current;
                current = list[i];
                count = 1;
            }
        }
        result += count;
        result += current;
        return result;
    }
```

```java
    public static String decompress(String s){
        String result = "";
        char[] list = s.toCharArray();

        if(!Character.isDigit(list[0]) || Character.isDigit(list[list.length - 1])){
            result = "Invalid Encoding";
            return result;
        }

        int count = 0;
        boolean prevDigit = false;

        for(int i = 0; i < list.length; i++){
            if(Character.isDigit(list[i])){
                if(prevDigit){
                    result = "Invalid Encoding";
                    return result;
                }
                count = list[i] - '0';
                prevDigit = true;
            }else{
                if(!prevDigit){
                    result = "Invalid Encoding";
                    return result;
                }

                for(int j = 0; j < count; j++){
                    result += list[i];
                }

                prevDigit = false;
            }
        }
        return result;
    }
}
```

Question 6 : Number Pattern Mirror Puzzle

Section 1 : Problem

The objective is to write a Java program that determines if two integer arrays, A and B, form a mirror pattern.

A mirror pattern is defined by the condition that the elements of array A must match the elements of array B in reverse order.

The program must:

1. Read two lines of comma-separated integers from the user.
2. Convert these inputs into two integer arrays.
3. Ensure both arrays have the same length (between 1 and 50).
4. Implement a method boolean isMirror(int[] a, int[] b) that returns true if they form a mirror pattern, and false otherwise.

Section 2 : Solution

The solution uses a do-while loop to enforce the length constraint and the String.split() method to convert the comma-separated string, combined with Integer.parseInt() to input string into integer arrays.

The core of the solution lies in the isMirror method, which determines if the elements of two integer arrays, A and B, match in reverse order. The fundamental mathematical relationship used is that for any index i in array A, the value A[i] must equal the value B[L - 1 - i], where L is the common length of the arrays.

Section 3 : Sample Input & Output

1st sample`

```
Array A: 2,4
Array B: 2,4,6
Please enter arrays with the same length.
Array A: 2,4,6
Array B: 6,4,2
Mirror pattern: true
```

2nd sample

```
Array A: 5,8,1,9,6,4
Array B: 5,6,9,1,8,4
Mirror pattern: false
BUILD SUCCESSFUL (total time: 18 seconds)
```

3rd sample

```
Array A: 1,2,3,4,5,6,7,8,9,10
Array B: 10,9,8,7,6,5,4,3,2,1
Mirror pattern: true
BUILD SUCCESSFUL (total time: 16 seconds)
```

Section 4 : Source Code

```
package viva02;

import java.util.Scanner;

public class Viva2Q6 {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

        int[] arrayA;
        int[] arrayB;

        do {
            System.out.print("Array A: ");
            String inputA = sc.nextLine();
            arrayA=parseArray(inputA);

            System.out.print("Array B: ");
            String inputB=sc.nextLine();
            arrayB=parseArray(inputB);

            if (arrayA.length != arrayB.length) {
                System.out.println("Please enter arrays with the same length.");
            }

        }while (arrayA.length != arrayB.length);

        System.out.println("Mirror pattern: "+isMirror(arrayA,arrayB));
    }
```

```
    public static boolean isMirror(int[]a,int[]b){
        if (a.length != b.length) {
            return false;
        }
        for (int i=0;i<a.length;i++){
            if (a[i]!=b[a.length-1-i]){
                return false;
            }
        }
        return true;
    }

    public static int[] parseArray(String line) {
        String[] parts = line.split(",");
        int[] arr = new int[parts.length];

        for (int i = 0; i < parts.length; i++) {
            arr[i] = Integer.parseInt(parts[i].trim());
        }
        return arr;
    }
}
```